MANUAL TÉCNICO
GESTOR DE ARCHIVOS



Javier Alejandro Mérida Gómez 202230638

23/10/2026

ÍNDICE

DESCRIPCIÓN GENERAL 2
OBJETIVOS 2
- Objetivo General 2
- Objetivos Específicos 2

Especificaciones del Sistema 4
- Requisitos de Hardware y Software 4
- Tecnologías 4
- Repositorio en GitHub 5

Instalación y Ejecución 5
- Variables de entorno 5
- Pasos para ejecutar el sistema 6

Arquitectura general 7
- Servidor (Node.js y Express) 7
- Cliente (Angular) 7
- Seguridad 7
- Persistencia con MongoDB 8

API REST 9
Modelo Documental 12
- Documentos embebidos y referencias 13
- Diagrama del modelo documental 14

Diagramas UML 15
- Link de los diagramas realizados en draw.io: 15
- Casos de uso 15
- Diagrama de casos de uso 22
- Diagrama de despliegue 23

DESCRIPCIÓN GENERAL

El Gestor de Archivos es un sistema de gestión documental que permite a los usuarios de una organización almacenar, organizar, consultar y compartir documentos de texto en formato .txt y .md. Reemplaza el manejo disperso de archivos en distintas computadoras por una plataforma central en la que cada documento tiene un propietario, una ubicación dentro de una estructura de carpetas y un control de acceso definido, y en la que queda registrado quién realizó cada operación y cuándo.

A nivel técnico, la solución sigue una arquitectura cliente-servidor: una aplicación web desarrollada en Angular consume una API REST construida con Node.js y Express, que es la única que se comunica con la base de datos MongoDB. El contenido de los archivos se guarda dentro de MongoDB mediante GridFS, y la información de usuarios, carpetas, documentos, permisos y actividad se modela como documentos, combinando datos embebidos y referencias según la forma en que el sistema los consulta.

OBJETIVOS

Objetivo General

Desarrollar un sistema de gestión documental con arquitectura cliente-servidor que permita almacenar, organizar, consultar y compartir archivos digitales entre usuarios, utilizando MongoDB como base de datos NoSQL orientada a documentos para administrar archivos, carpetas, usuarios, permisos y la actividad realizada en el sistema.

Objetivos Específicos

- **Diseñar el modelo documental:** definir las colecciones, los documentos embebidos, las referencias y los índices de MongoDB según las consultas que realiza el sistema.
- **Implementar la arquitectura cliente-servidor:** construir una aplicación en Angular y un servidor en Node.js independientes, comunicados

mediante una API REST.

- **Almacenar el contenido en MongoDB:** guardar el contenido de cada archivo con GridFS y recuperarlo posteriormente sin pérdida de información.
- **Aplicar las reglas de negocio:** mantener la consistencia entre carpetas, archivos, permisos, espacio de almacenamiento y eliminación de recursos.
- **Generar reportes:** ofrecer búsquedas, filtros y reportes con agregaciones sobre archivos, usuarios, comparticiones y actividad.

Especificaciones del Sistema

Requisitos de Hardware y Software

- Sistemas operativos compatibles: Microsoft Windows 10 (o superior), GNU/Linux (Ubuntu 20.04 LTS o superior).
- Node.js 22 o superior con npm.
- MongoDB 8 en ejecución local en el puerto 27017.
- Navegador web.
- Entorno de desarrollo: Visual Studio Code.

Tecnologías

Desarrollo	Tecnología / Librería	Función
Interfaz gráfica	Angular 21.2	Construcción de la interfaz web (SPA) que consume la API REST del servidor.
	TypeScript 5.9	Lenguaje de la aplicación cliente.
	RxJS 7.8	Manejo de las peticiones HTTP asíncronas hacia el servidor.
Backend	Node.js 22	Entorno de ejecución del servidor.
	Express 5.3	Framework que expone la API REST y organiza las rutas y los middlewares.
	Mongoose 9.11	Definición de los modelos, validaciones y consultas a MongoDB.
	jsonwebtoken 9	Generación y verificación de los tokens JWT de la sesión.
	bcryptjs 3	Cifrado de las contraseñas antes de guardarlas.
	multer 2.4	Recepción de los archivos que se suben desde el cliente.

	chardet 2.2 e iconv-lite 0.7	Detección de la codificación de los archivos subidos y conversión a UTF-8.
	cors y dotenv	Permitir las peticiones desde Angular y leer la configuración del archivo .env.
Persistencia de datos	MongoDB 8.2	Base de datos NoSQL orientada a documentos donde se guarda toda la información del sistema.
	GridFS	Almacenamiento del contenido de los archivos dentro de MongoDB, dividido en fragmentos.

Repositorio en GitHub

Link del repositorio: https://github.com/JavierMeridaLk/ProyectoDosMIA-SS-26

Instalación y Ejecución

Variables de entorno

El servidor lee su configuración de un archivo .env ubicado en la carpeta servidor-app. El repositorio incluye el archivo .env.example como plantilla con las siguientes variables:

Variable	Descripción	Ejemplo
PORT	Puerto en el que se ejecuta el servidor.	3001
MONGO_URI	Cadena de conexión a la base de datos.	mongodb://localhost:27017/gestor_archivos
JWT_SECRET	Clave con la que se firman los tokens de sesión.	Cadena larga y aleatoria
JWT_EXPIRA	Tiempo de vigencia de la sesión.	8h
ESPACIO_PREDETERMINADO	Espacio asignado a cada usuario nuevo, en bytes.	1073741824 (1 GB)

ADMIN_USER, ADMIN_CORREO y ADMIN_CONTRASENA	Datos del administrador inicial que crea el script seed:admin.	admin, admin@gestor.com

Pasos para ejecutar el sistema

1. Iniciar el servicio de MongoDB en el equipo.
2. En la carpeta servidor-app, copiar .env.example como .env y ajustar los valores si es necesario.
3. En la carpeta servidor-app, instalar las dependencias con el comando npm install.
4. Cargar la carga inicial por medio del script creado.
5. Iniciar el servidor con npm run dev (o npm start). Queda disponible en http://localhost:3001.
6. En la carpeta gestor-archivos, instalar las dependencias con npm install e iniciar la aplicación con npm start (o ng serve).
7. Abrir http://localhost:4200 en el navegador.

Arquitectura general

El sistema sigue una arquitectura cliente-servidor de tres capas: una aplicación cliente en Angular que se ejecuta en el navegador, una API REST en Node.js con Express que concentra la lógica de negocio y es la única que accede a la base de datos, y MongoDB como capa de persistencia.

El cliente envía en cada petición el token de sesión, y el servidor verifica la identidad y los permisos del usuario antes de leer o modificar cualquier recurso.

Servidor (Node.js y Express)

- **routes:** definen los endpoints de cada módulo y los middlewares que se aplican a cada uno.
- **middlewares:** verificarToken valida el JWT y que el usuario siga activo, soloAdmin restringe las rutas de administración y recibirArchivo procesa la subida de archivos con un límite de 5 MB.
- **controllers:** reciben cada petición, validan los datos y arman la respuesta.
- **services:** contienen la lógica compartida: permisos sobre recursos, acceso a GridFS, detección de codificación, registro de actividad y búsqueda de texto.
- **models:** esquemas de Mongoose de cada colección.

Cliente (Angular)

- **core:** modelos, servicios HTTP, guards de rutas y el interceptor que agrega el token a cada petición.
- **pages:** pantallas inicio de sesión, Mi Unidad, editor, compartidos, papelera, búsqueda y administración.
- **shared:** componentes reutilizables, como los iconos, los diálogos de confirmación y la lista de actividad.

Seguridad

- Las contraseñas se cifran con bcrypt antes de guardarse y nunca se devuelven en las respuestas.
- La sesión se maneja con JSON Web Token, con una vigencia configurable (8

horas por defecto).

- Existen dos roles, USUARIO y ADMIN. Las rutas de usuarios, logs y reportes solo responden al administrador.
- Los permisos sobre cada recurso se verifican en el servidor en cada operación. Un permiso sobre una carpeta se hereda a todo su contenido.
- Un usuario desactivado no puede iniciar sesión y su token deja de ser válido, pero su información se conserva.

Persistencia con MongoDB

- Cinco colecciones con validadores $jsonSchema: usuarios, carpetas, documentos, comparticiones y logs.
- El contenido de los archivos se guarda con GridFS en el bucket archivos (colecciones archivos.files y archivos.chunks), y cada documento guarda la referencia en gridfsId.
- Todo archivo se almacena en UTF-8. Al subirlo se detecta su codificación original (UTF-16, Windows-1252, ISO-8859-1, entre otras) y se convierte.
- Las operaciones que necesitan agrupar o acumular datos, como los reportes y el espacio usado, se resuelven con pipelines de agregación de MongoDB.

API REST

Todas las rutas usan el prefijo /api. Las respuestas se envían en formato JSON.

Método	Ruta	Descripción
POST	/auth/registro	Registra un usuario con rol USUARIO.
POST	/auth/login	Inicia sesión y devuelve el token.
GET	/auth/perfil	Datos del usuario autenticado.
GET	/carpetas/contenido	Contenido de Mi Unidad.
GET	/carpetas/:id/contenido	Contenido de una carpeta propia o compartida.
GET	/carpetas/arbol	Carpetas propias, para elegir el destino al mover.
POST	/carpetas	Crea una carpeta.
PATCH	/carpetas/:id/nombre	Renombra una carpeta.
PATCH	/carpetas/:id/mover	Mueve una carpeta evitando ciclos.
DELETE	/carpetas/:id	Envía una carpeta y su contenido a la papelera.
GET	/documentos/almacenamiento	Espacio usado y asignado.
POST	/documentos	Crea un documento vacío.
POST	/documentos/subir	Sube un archivo .txt o .md.
GET	/documentos/:id	Contenido, comentarios y permiso del usuario.
PUT	/documentos/:id/contenido	Guarda los cambios del documento.
GET	/documentos/:id/descargar	Descarga el archivo.
POST	/documentos/:id/comentarios	Agrega un comentario.

PATCH	/documentos/:id/nombre	Renombra un documento.
PATCH	/documentos/:id/mover	Mueve un documento a otra carpeta.
DELETE	/documentos/:id	Envía un documento a la papelera.
GET	/papelera	Elementos en la papelera.
POST	/papelera/:tipo/:id/restaurar	Restaura un elemento a su ubicación.
DELETE	/papelera/:tipo/:id	Elimina un elemento de forma definitiva.
GET	/comparticiones/conmigo	Recursos que otros usuarios me compartieron.
GET	/comparticiones/usuarios	Sugerencias de usuarios para compartir.
GET	/comparticiones/:tipo/:id	Usuarios con acceso a un recurso.
POST	/comparticiones	Comparte un recurso con uno o varios usuarios.
PATCH	/comparticiones/:id	Cambia el permiso de una compartición.
DELETE	/comparticiones/:id	Retira el acceso a un usuario.
GET	/busqueda	Busca carpetas y documentos con filtros.
GET	/usuarios	Lista de usuarios con su espacio usado (admin).
PATCH	/usuarios/:id/rol	Cambia el rol de un usuario (admin).
PATCH	/usuarios/:id/activo	Activa o desactiva un usuario (admin).
PATCH	/usuarios/:id/almacenamiento	Cambia el espacio asignado (admin).

GET	/actividad	Historial de actividad con filtros (admin).
GET	/reportes/:reporte	Genera cada uno de los reportes (admin).

Modelo Documental

La información se organiza en las siguientes colecciones. Los datos que siempre se leen junto con su documento se guardan embebidos, y las entidades que se consultan o modifican por separado se relacionan mediante referencias.

Colección	Contenido y documentos embebidos	Referencias	Índices
usuarios	Usuario, correo, contraseña cifrada, rol, estado activo, espacio asignado y fecha de creación.	Ninguna	user (único), correo (único)
carpetas	Nombre, ruta (arreglo de antecesores), fecha de creación y eliminacion embebido (enPapelera, fecha, origenId).	propietarioId a usuarios, padreId a carpetas	propietarioId + padreId, ruta
documentos	Nombre, extensión, tamaño, fechas de carga y modificación, eliminacion embebido y arreglo de comentarios embebidos.	propietarioId a usuarios, carpetaId a carpetas, gridfsId a archivos.files	propietarioId + carpetaId, nombre (texto), fechaCarga
comparticiones	Tipo de recurso, permiso (LECTURA, COMENTAR o EDICION) y fecha.	recursoId a documentos o carpetas, propietarioId y usuarioId a usuarios	usuarioId, recursoId + usuarioId (único)
logs	Acción, fecha, detalle, nombre del usuario y copia embebida del recurso (id, tipo y nombre).	usuarioId y propietarioId a usuarios	usuarioId + fecha, accion, propietarioId + fecha, recurso.id
archivos.files y archivos.chunks	Contenido de cada archivo dividido en fragmentos (GridFS).	metadata.propietarioId a usuarios	Índices propios de GridFS

Documentos embebidos y referencias

- **Datos de eliminación embebidos:** el estado de papelera se consulta en cada lectura de carpetas y documentos, por eso vive dentro de cada uno. El campo origenId indica qué carpeta envió el elemento a la papelera, para restaurarlo o eliminarlo junto con ella.
- **Comentarios embebidos:** siempre se muestran junto con su documento y son pocos por documento. Guardan el nombre del autor para no consultar la colección de usuarios en cada lectura.
- **Arreglo de antecesores:** cada carpeta guarda los ids de todas sus carpetas superiores. Permite armar la barra de navegación sin consultas recursivas y obtener todos los descendientes de una carpeta con una sola consulta indexada, lo que se usa al mover, eliminar y compartir carpetas, y para evitar ciclos.
- **Copia del recurso en los logs:** el historial guarda el id, el tipo y el nombre del recurso en el momento de la acción, de modo que se conserva aunque el recurso se renombre o se elimine definitivamente.
- **Referencias:** usuarios, carpetas, documentos y comparticiones se crean y modifican por separado, por lo que se relacionan por id para no duplicar datos que cambian.

Diagrama del modelo documental

Diagramas UML

Link de los diagramas realizados en draw.io:

https://drive.google.com/file/d/1PDssBRSlW9trxvWhdIvNS0zpoCHI4xCV/view?usp=sharing

Casos de uso

Número:	**CU01.**
Caso de Uso:	Registrarse
Actores:	Usuario
Descripción:	El usuario ingresa su correo, nombre de usuario, contraseña y la confirmación de la contraseña. El sistema valida el formato del correo, que la contraseña tenga al menos 4 caracteres y coincida con la confirmación, y que el usuario y el correo no estén registrados. Cifra la contraseña, crea la cuenta con rol USUARIO y el espacio predeterminado, registra la acción en el historial e inicia la sesión.
Tipo:	Primario

Número:	**CU02.**
Caso de Uso:	Iniciar sesión
Actores:	Usuario, Administrador
Descripción:	El usuario ingresa su nombre de usuario y contraseña. El sistema valida las credenciales contra la contraseña cifrada y verifica que la cuenta esté activa. Si son correctas, genera un token JWT, guarda la sesión en el navegador, registra el inicio de sesión y muestra Mi Unidad. Al administrador además le muestra la sección de administración.
Tipo:	Primario

Número:	**CU03.**
Caso de Uso:	Crear carpeta
Actores:	Usuario
Descripción:	Desde Mi Unidad o una carpeta propia, el usuario selecciona "Nueva Carpeta" e ingresa el nombre. El sistema valida que el nombre no esté vacío, que no contenga / ni \ y que no exista otra carpeta con el mismo nombre en esa ubicación. Calcula la ruta de antecesores a partir de la carpeta padre, guarda la carpeta y registra la acción.
Tipo:	Primario

Número:	**CU04.**
Caso de Uso:	Crear documento
Actores:	Usuario
Descripción:	El usuario selecciona "Nuevo Documento", ingresa el nombre y elige el tipo (.txt o .md). El sistema verifica que tenga espacio disponible, guarda el contenido vacío en GridFS, crea el documento en la carpeta actual, registra la acción y abre el editor.
Tipo:	Primario

Número:	**CU05.**
Caso de Uso:	Subir archivo
Actores:	Usuario
Descripción:	El usuario selecciona "Subir Archivo" y elige un archivo .txt o .md de hasta 5 MB. El sistema valida la extensión, detecta la codificación del archivo y lo convierte a UTF-8, rechaza los archivos binarios, verifica el espacio disponible, guarda el contenido en GridFS y registra la acción.

Tipo:	Primario

Número:	**CU06.**
Caso de Uso:	Editar documento
Actores:	Propietario, usuario con permiso de edición
Descripción:	El usuario abre el documento en el editor, modifica el contenido y presiona "Guardar" (o Ctrl+S). El sistema verifica el permiso de edición y el espacio del propietario, guarda la nueva versión en GridFS, elimina la anterior, actualiza el tamaño y la fecha de modificación y registra la acción.
Tipo:	Primario

Número:	**CU07.**
Caso de Uso:	Descargar documento
Actores:	Propietario, usuario con acceso
Descripción:	Desde el editor, el usuario presiona "Descargar". El sistema verifica que tenga al menos permiso de lectura, devuelve el archivo con su nombre y extensión y registra la descarga.
Tipo:	Secundario

Número:	**CU08.**
Caso de Uso:	Renombrar recurso
Actores:	Propietario
Descripción:	El propietario abre el menú de una carpeta o documento y elige "Renombrar". El sistema valida el nuevo nombre, conserva la extensión del documento, evita nombres de carpeta repetidos en la misma ubicación y registra el nombre anterior en el historial.
Tipo:	Secundario

Número:	**CU09.**
Caso de Uso:	Mover recurso
Actores:	Propietario
Descripción:	El propietario elige "Mover" y selecciona la carpeta destino en el árbol de carpetas. El sistema impide moverlo a la ubicación donde ya está y mover una carpeta dentro de sí misma o de una de sus subcarpetas. Actualiza la ruta de la carpeta y de todos sus descendientes y registra el origen y el destino.
Tipo:	Secundario

Número:	**CU10.**
Caso de Uso:	Compartir recurso
Actores:	Propietario
Descripción:	El propietario elige "Compartir", indica uno o varios usuarios por nombre de usuario o correo y selecciona el permiso (Lectura, Comentar o Edición). El sistema verifica que cada usuario exista, esté activo y no sea el propietario, y que el recurso no esté en la papelera. Crea o actualiza la compartición y la registra. Compartir una carpeta da acceso a todo su contenido.
Tipo:	Primario

Número:	**CU11.**
Caso de Uso:	Modificar o retirar permisos
Actores:	Propietario
Descripción:	En la lista de personas con acceso de un recurso, el propietario cambia el permiso de un usuario o le retira el acceso. Solo el propietario puede hacerlo. El cambio tiene efecto inmediato y queda registrado en el historial.
Tipo:	Secundario

Número:	**CU12.**
Caso de Uso:	Comentar documento
Actores:	Propietario, usuario con permiso de comentar o edición
Descripción:	El usuario abre el panel de comentarios del documento, escribe un comentario y presiona "Comentar". El sistema verifica el permiso, guarda el comentario dentro del documento con su autor y fecha, y registra la acción.
Tipo:	Secundario

Número:	**CU13.**
Caso de Uso:	Enviar a la papelera
Actores:	Propietario
Descripción:	El propietario elige "Mover a la papelera" sobre una carpeta o documento. El recurso deja de mostrarse en las consultas normales. Si es una carpeta, sus subcarpetas y documentos se envían junto con ella. Los usuarios con quienes estaba compartido pierden el acceso mientras esté en la papelera.
Tipo:	Primario

Número:	**CU14.**
Caso de Uso:	Restaurar o eliminar definitivamente
Actores:	Propietario
Descripción:	En la papelera, el propietario restaura un elemento o lo elimina de forma definitiva. Al restaurar, el elemento vuelve a su ubicación original, o a Mi Unidad si su carpeta ya no existe. Al eliminar definitivamente, previa confirmación, se borran el recurso, su contenido en GridFS y sus compartidos, liberando el espacio.
Tipo:	Primario

Número:	**CU15.**
Caso de Uso:	Buscar archivos y carpetas
Actores:	Usuario
Descripción:	El usuario escribe en la barra de búsqueda. El sistema busca las carpetas y documentos cuyo nombre contenga el texto, sin importar mayúsculas ni acentos, entre los recursos propios y los compartidos, y permite filtrar por tipo, rango de fechas y autor.
Tipo:	Primario

Número:	**CU16.**
Caso de Uso:	Administrar usuarios
Actores:	Administrador
Descripción:	El administrador consulta la lista de usuarios con su espacio usado, cambia el rol, activa o desactiva cuentas y modifica el espacio asignado. No puede modificar su propio rol ni desactivar su propia cuenta. Cada cambio queda registrado en el historial.
Tipo:	Primario

Número:	**CU17.**
Caso de Uso:	Consultar logs
Actores:	Administrador
Descripción:	El administrador consulta el historial de actividad de todo el sistema y lo filtra por usuario, acción, tipo de recurso y rango de fechas.
Tipo:	Secundario

Número:	**CU18.**
Caso de Uso:	Consultar reportes
Actores:	Administrador
Descripción:	El administrador elige un reporte de los grupos Archivos y almacenamiento, Usuarios, Compartición o Actividad. El sistema lo genera con agregaciones de MongoDB y lo muestra en una tabla numerada. Algunos reportes piden un usuario, un rango de fechas o un período.
Tipo:	Primario

Diagrama de casos de uso

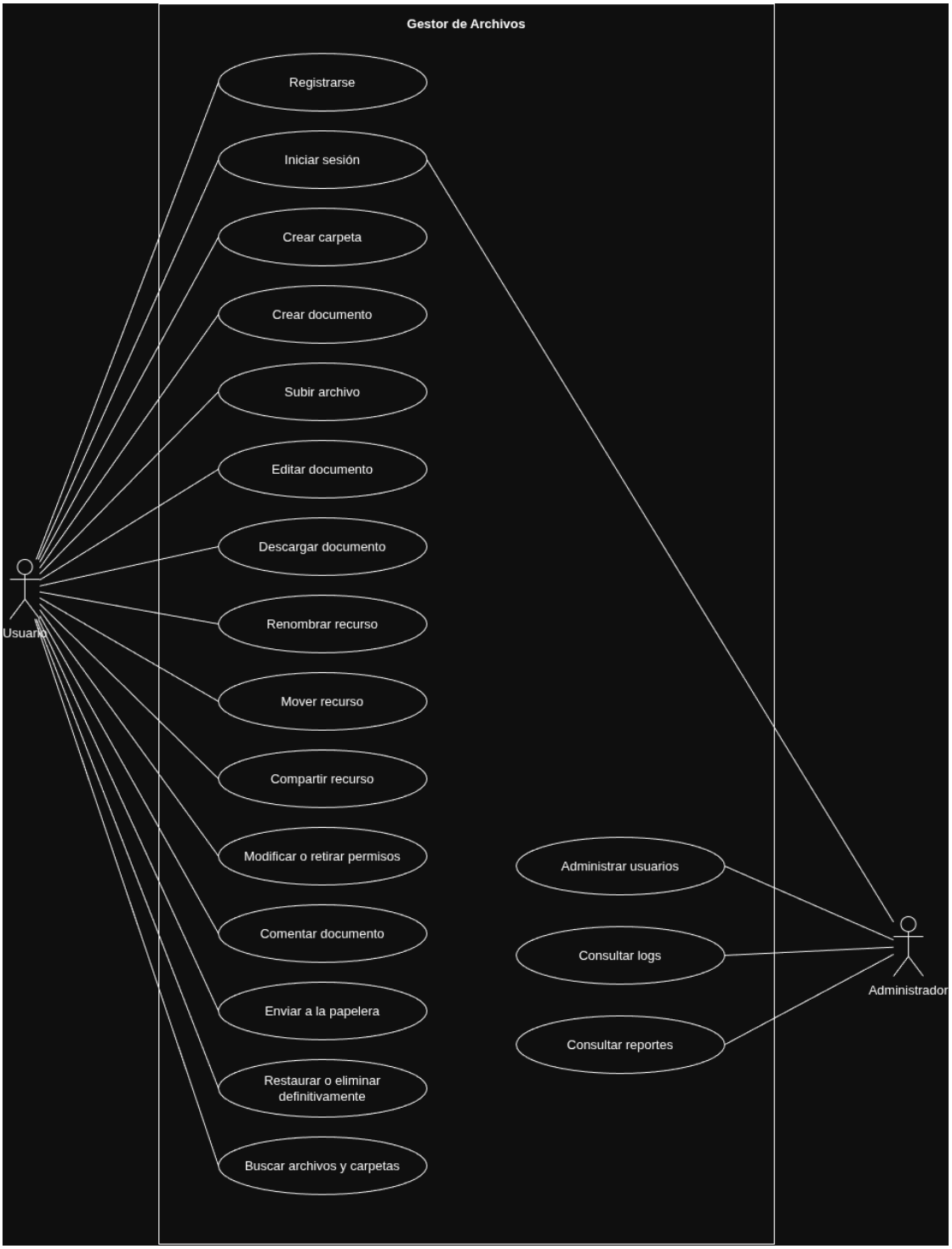


Diagrama de despliegue

